WeatherManager Splitup (for **S**ingle responsibility in **S**OLID)

Low cohesion in WeatherManager since it has too many responsibilities

Figure 1. Low Cohesion vs High Cohesion Concept Diagram

Originally, WeatherManager performs too many tasks (hence **low cohesion)**:

- Answers queries on isDamaging, isObscuring
- Applies lightningRod attraction + calculates the radius susceptible to lightning
- Adds weather spawn points (for clouds and lightning)
- Ticks all weather
- Renders all weather

Consequently, it is decided that WeatherManager be split into three classes:

WeatherSpawnManager: only addSpawnPoint and getSpawnPoints (ticked)

LightningManager: only handle attractLightning

WeatherQueryManager: only handle isObscuring and isDamaging

With WeatherManager calling these classes for their modular portion of work in the existing methods.

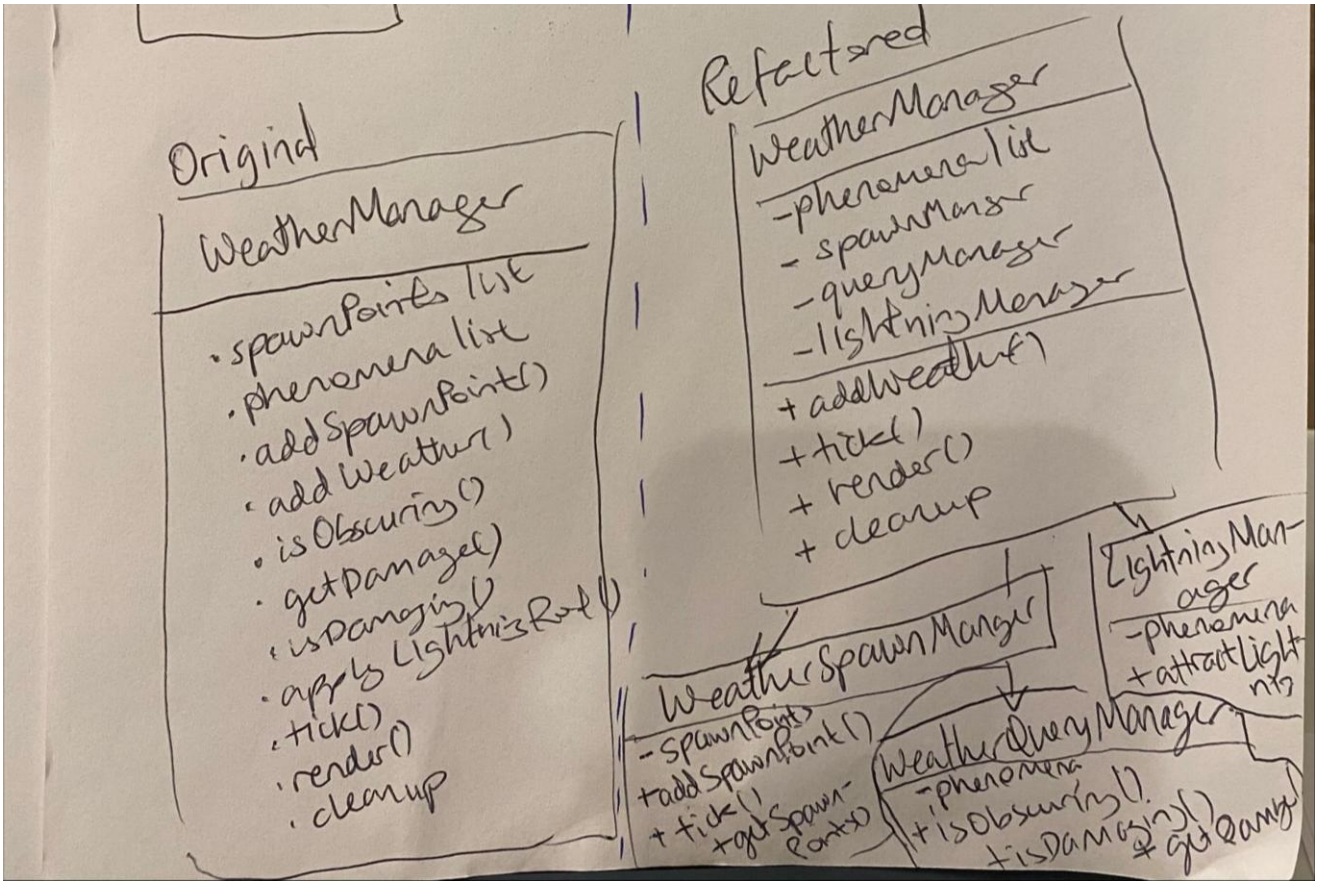


Figure 2. Class Diagram of WeatherManager before and after refactoring

--

Even after splitting weather, however, there are many other design issues to refactor here.

--

Open/closed principle (O in SOLID)

First, *the instanceof Lightning* check violates the **Open/closed principle**. Adding new weather types would require modifying this existing method.

```
 */
public void attractLightning(Positionable position) {  1 usage   p
    for (GameEntity weather : phenomena) {
        if (weather instanceof Lightning) {
            final Lightning bolt = (Lightning) weather;
            int deltaX = position.getX() - bolt.getX();
            int deltaY = position.getY() - bolt.getY();
            final int distance = (int) Math.sqrt(deltaX * deltaX
            if (distance <= LightningRod.RADIUS) {
                bolt.setX(position.getX());
                bolt.setY(position.getY());
            }
```

Figure 3. Original method before refactors

To fix, a new class Attractable captures all lightning attracting items in consideration of future machines that could attract lightning. Now, we simply call for all Attractable weather entities, skipping the non-attractables using polymorphism.

```
    public void attractLightning(Positionable position) {  1 usage   porsce *
        for (GameEntity weather : phenomena) {
            if (!(weather instanceof Attractable)) {
                continue; // skip non-attractable weather
            }
            int deltaX = position.getX() - weather.getX();
            int deltaY = position.getY() - weather.getY();
            final int distance = (int) Math.sqrt(deltaX * deltaX + deltaY * deltaY);

            if (distance <= LightningRod.RADIUS) {
                weather.setX(position.getX());
                weather.setY(position.getY());
            }
        }
    }
```

Figure 4. Attractable class created and used

To further improve and completely adhere to the **Open/closed principle**, I considered passing in int radius to the method so that it can be extendable for new machines with specific radius of attraction; however, this is currently not doable because some of the provided tests require the current exact method signature (i.e. without radius) in Weather.java, see Figures 6 and 7. Nevertheless, this idea is documented for evidence of thinking.

If I were not constrained by the tests, below is what I would change the method signature to:

```
void applyLightningRod(Positionable position, int radius);
```

where subsequent method calls become like below, in LightningRod.java:

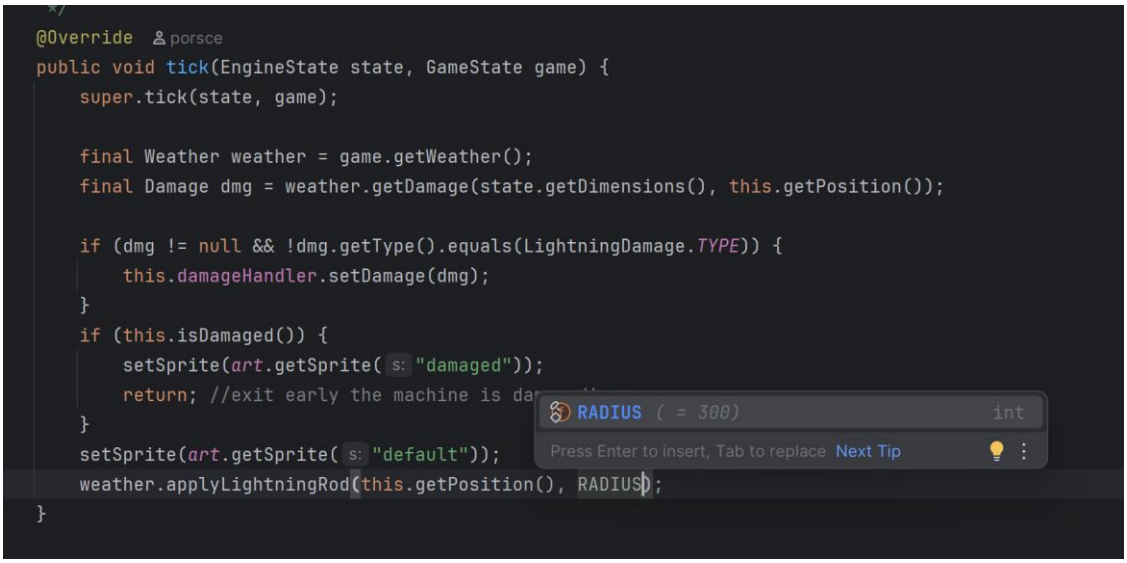


Figure 5. Calling applyLightningRod with RADIUS in LightningRod.java

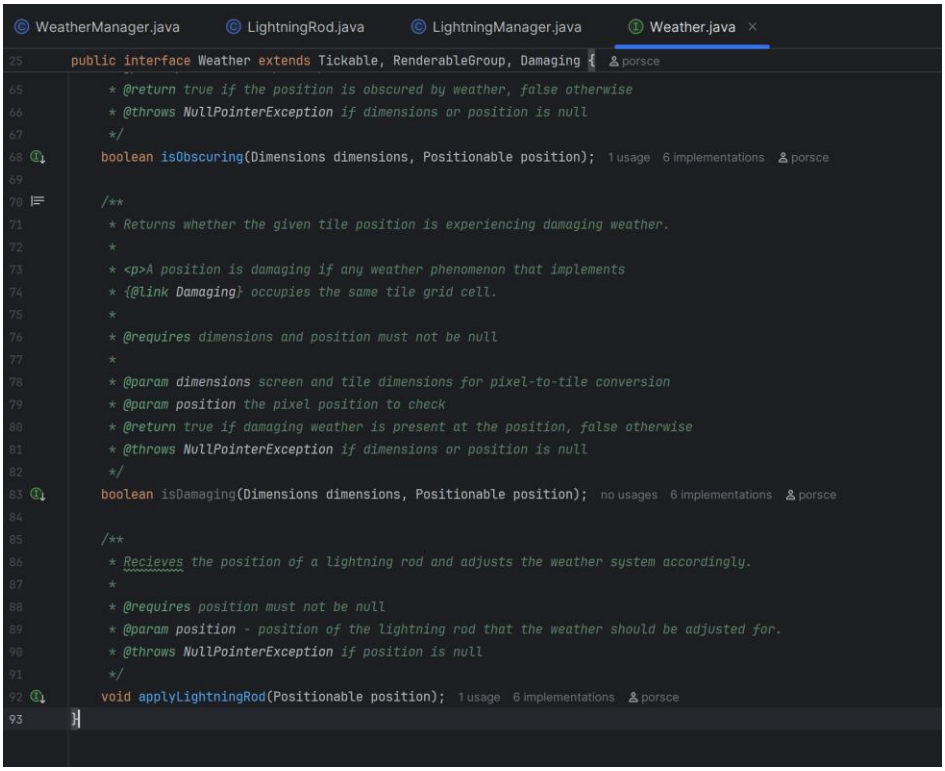


Figure 6. Weather interface uses applyLightningRod with 6 implementations

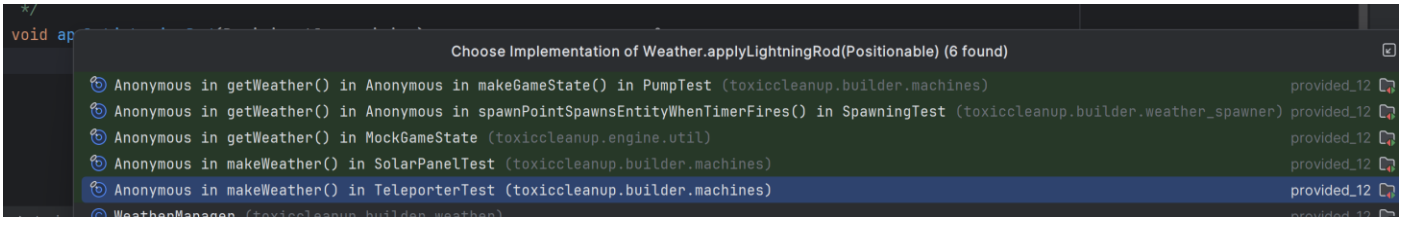
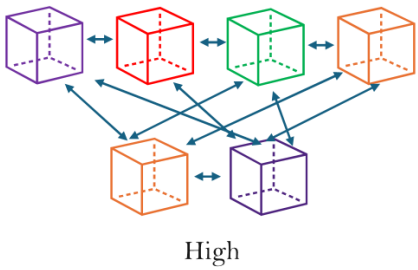


Figure 7. applyLightningRod method signature cannot be changed due to these tests' implementations

Tight Coupling



High coupling between LightningManager and Lightning due to strong interconnections where changing one creates issues for the other.

Figure 8. High Coupling Concept Diagram

Second, there is **tight coupling** between LightningManager and Lightning via the direct position modification of bold.setX and bolt.setY as highlighted yellow below.

```
 */
public void attractLightning(Positionable position) {  1 usage  p
    for (GameEntity weather : phenomena) {
        if (weather instanceof Lightning) {
            final Lightning bolt = (Lightning) weather;
            int deltaX = position.getX() - bolt.getX();
            int deltaY = position.getY() - bolt.getY();
            final int distance = (int) Math.sqrt(deltaX * deltaX
            if (distance <= LightningRod.RADIUS) {
                bolt.setX(position.getX());
                bolt.setY(position.getY());
            }
```

Figure 9. identified Feature Envy on Lightning

Feature Envy (Code smell)

This is a code smell too, as LightningManager seems more interested in another class, Lightning's data than its own here. To fix, the set position task is delegated to Lightning, which is implemented using the interface attractTo method present in Attractable too.

```java
package toxiccleanup.builder.weather;

import toxiccleanup.engine.game.Positionable;

/**
 * Interface for weather phenomena that can be attracted by lightning rods.
 */
public interface Attractable {  2 usages  1 implementation  new *
    /**
     * Attracts this weather phenomenon to the given position.
     *
     * @param targetPosition the position to attract to
     * @return true if the phenomenon was moved, false otherwise
     */
    boolean attractTo(Positionable targetPosition);  no usages  1 implementation  new *
}
```

Figure 10. Attractable interface

Implementation moved to Lightning's attractTo:

```java
@Override  no usages  new *
public boolean attractTo(Positionable targetPosition) {
    // Lightning can be attracted
    setX(targetPosition.getX());
    setY(targetPosition.getY());
    return true;
}
```

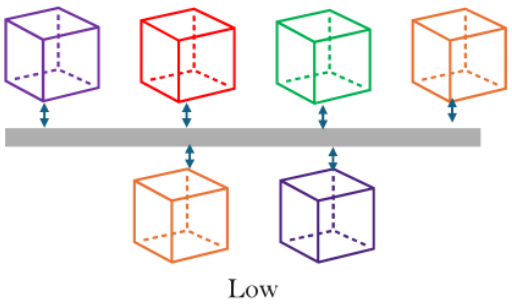
Figure 11. moved Feature Envy part to Lightning itself

Now the LightningManager is adapted to:

```java
public void attractLightning(Positionable position) {  1 usage  porsce *
    for (GameEntity weather : phenomena) {
        if (!(weather instanceof Attractable)) {
            continue; // skip non-attractable weather
        }
        int deltaX = position.getX() - weather.getX();
        int deltaY = position.getY() - weather.getY();
        final int distance = (int) Math.sqrt(deltaX * deltaX + deltaY * deltaY);

        if (distance <= LightningRod.RADIUS) {
            ((Attractable) weather).attractTo(position);
        }
    }
}
```

Figure 12. refactored LightningManager



So that coupling is reduced! :)

Figure 13. Low Coupling Concept Diagram

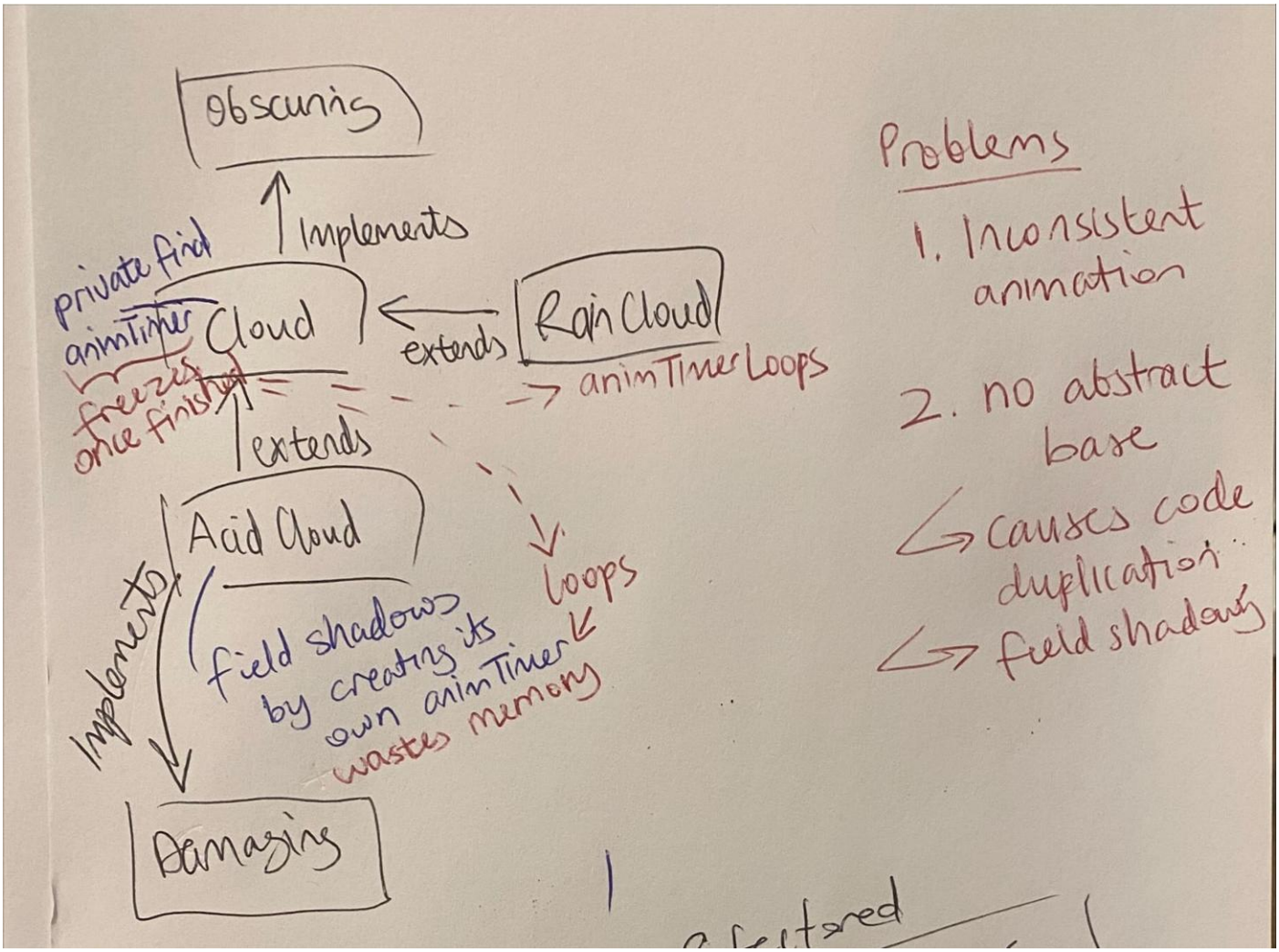
Unused Cleaning Interface - removed

In the provided code, Cleaning interface has no methods or javadocs, with no clear intention on how it's to be used. There are also no tests running on it, so this class has been removed.

Unused Disableable Interface – removed

Similar to the Cleaning interface, Disableable has no methods or javadocs, with no clear intention on how it's to be used. There are also no tests running on it, so this class has been removed.

Cloud Inheritance



All cloud classes (Cloud, RainCloud, AcidCloud) share a bit too many things:

- Movement logic (timer, speed, direction)
- Animation logic (timer, frame cycling)
- Boundary detection (remove when x < 0)
- Obscuring interface (obscure solarpanels)

This calls for a refactor!

Hence, two options were considered: abstract class and interface.

Abstract classes are less flexible (if clouds need to extend something else) than interfaces. However, if an interface was chosen, there would be *massive code duplication* as each cloud would re-implement movement, animation, boundary, etc.

Since interfaces cannot hold shared state (even timers would be duplicated, which may cause inconsistencies), I decided to use an **abstract class** here called **Cloudable**. This class will replace the Cloud class's purpose, but as an abstract class that only extends Obscuring, as seen below. Its inheritors will be Cloud, AcidCloud, and RainCloud, where each of them would extend Cloudable.

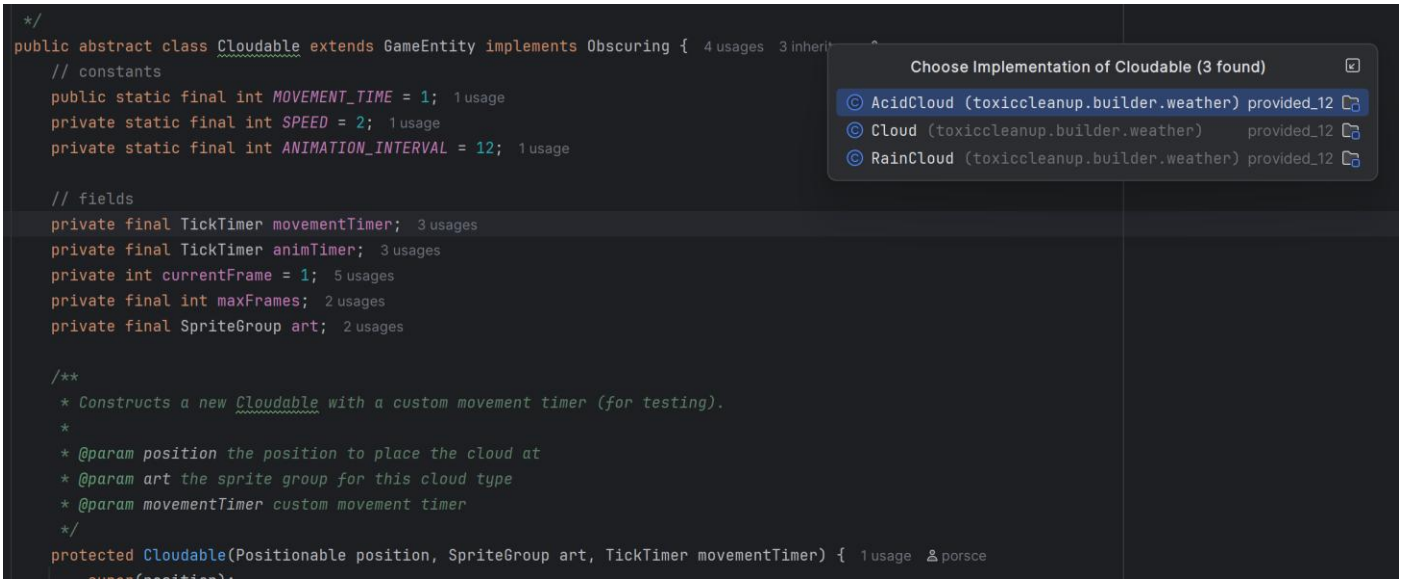


Figure 14. Cloudable has three inheritors: AcidCloud, Cloud, RainCloud

AcidCloud extends Cloudable implements Damaging

Cloud extends Cloudable

RainCloud extends Cloudable

So by chain inheritance, every cloud implements Obscuring (they obscure the sun afterall).

Code smells

Inconsistent Logic

Cloud.java used to have a fixed implementation that freezes cloud movement once it reaches boundary, which neither RainCloud nor AcidCloud do. Hence, upon refactoring all clouds are removed cleanly once they've moved across the screen.

Duplication

Boundary checking before marking for removal is duplicated in every cloud class. This is bad, because fixing one means fixing all. Hence, it was moved to the Cloudable mother class and now occurs just once and for all.

```
/**
 * Checks if cloud has moved off-screen and marks for removal if so.
 */
private void checkBoundary() {  1 usage  porsce
    if (getX() < 0) {
        markForRemoval();
    }
}
```

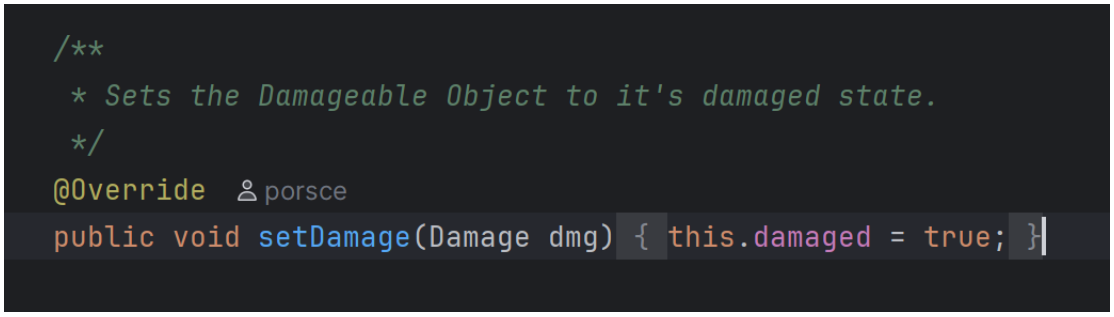
Likewise, Cloud, RainCloud and AcidCloud all had their own animation fields like currentArtFrame and animTimer. This wastes memory, can cause inconsistent behaviour and is difficult to mass-alter if the timer is suddenly changed due to client requests.

```
private int currentArtFrame = 1;

private final TickTimer animTimer = new RepeatingTimer(12);
```

Bad Naming

1. Changed to: private final TickTimer **movementTimer**;
   instead of: private final TickTimer **timer**;
   Since this is less clear what timer this is (there's already an animTimer)
2. Misleading method name: setDamage suggets it stores some Damage object, but it only sets a boolean. A better name would be makeDamaged to meaningfully communicate the object getting this method will be damaged. However, this change cannot be done as the provided tests will not pass.



3. adjust method is not descriptive enough, many variations of this in different classes.

```
public void adjust(int amount) {
   hp -= amount;
   hp = Math.clamp(hp, 0, MAX_HP);
```

Above is playerManager's adjust, which changes hp. Below is machinesManager's adjust, which changes power instead.

```
public void adjust(int amount) {
   this.gainPower(amount);
}
```

A proposal would be to call them adjustHp and adjustPower; however refactoring playerManager is beyond scope of A2.

SpawnFactory

Open/Closed Principle (O from SOLID)

By changing the if statement to a switch, adding a new weather type only requires adding one new case to the switch instead of modifying the existing if-else chain (which added 2 branches when this could be done in 1 in the new implementation).

"Software entities should be open for extension, but closed for modification."

Repetition Removed

Instead of repeating lowercase and uppercase cases for each letter, it was handled in one place using logic:

```
boolean isUppercase =
Character.isUpperCase(symbol);

int finalSpawnTime = isUppercase

  ? (int)(baseSpawnTime *
MULTIPLIER)

  : baseSpawnTime;
```

With finalSpawnTime multiplied to each case's spawning instead of a statically hardcoded number for each. This simplifies the reading, logic and reduces code duplication.

Information Hiding (to avoid magic numbers)

In SpawnerFactory, a field was created: private static final double **SPAWN_MULTIPLIER** = 5.5;

WeatherSpawnPoint

Original: private final **ArrayList**<Positionable> positions = new ArrayList<>();

New: private final Positionable position;

For some reason, the position is stored as an array list when there is only **one** position for each spawned weather point. This can mislead developers as it suggests positions might exist, but they don't. Further, on a memory usage perspective it is wasteful to do this.

Conveniently, this simplifies some code in WeatherSpawnPoint, e.g. directly access the point in getPosition and WeatherSpawnPoint:

```
 */
public Positionable getPosition() {  porsce *
    return new Position(position.getX(), position.getY());
}

public WeatherSpawnPoint(Positionable position, TickTimer timer, S
    this.position = position;
    this.timer = timer;
    this.spawner = spawner;
}
```

LightningDamage's Fields Shadowed Parent

Removed:

private int x = 0;
private int y = 0;

Since parent, Damage, already has x and y defined, LightningDamage inherits them via super(position). The child's x and y were **shadowing** the parent's fields, which can cause bugs.

```
public class Damage implements Positionable {
   private int x = 0;
   private int y = 0;
   // ...
   public Damage(Positionable position) {
      this.x = position.getX();
      this.y = position.getY();
   }
}
```

Defensive programming: instead of adding it to a final array list,

return new ArrayList<>(phenomena);

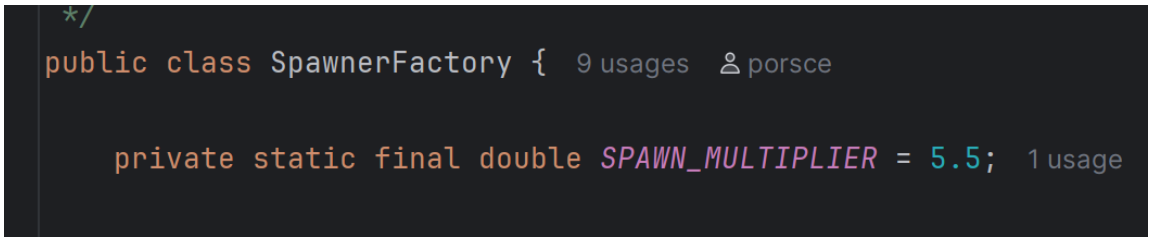
```java
/**
 * A collection of renderables that should each be displayed.
 *
 * @return A collection of renderables to display.
 */
@Override  porsce
public List<Renderable> render() {
    final ArrayList<Renderable> renderables = new ArrayList<>();
    renderables.addAll(phenomena);
    return renderables;
}
```

Magic Numbers

1. In SpawnerFactory, the 5.5 appeared a catastrophic number of times, usually through the many

   *new RepeatingTimer((int) (Cloud.SPAWN_TIME * 5.5))*

   in the massive if statements. This was reduced by adding the field:



2. In Cloud.java, the 12:

   private final TickTimer animTimer = new RepeatingTimer(12);

   Removed by adding field in Cloudable which got inherited by all clouds:

   private static final int ANIMATION_INTERVAL = 12;

3. In Lightning.java, the 5:

   if (animFrame == 5 || animFrame == 6)

   removed by adding

   private static final int DAMAGE_FRAME_START = 5;
   private static final int DAMAGE_FRAME_END = 6;

   so that code becomes:
   animFrame == DAMAGE_FRAME_START || animFrame == DAMAGE_FRAME_END;